«Нельзя дважды войти в одну и ту же реку.» — *Гераклит*

1. Карты Карно

a) Генерация функций

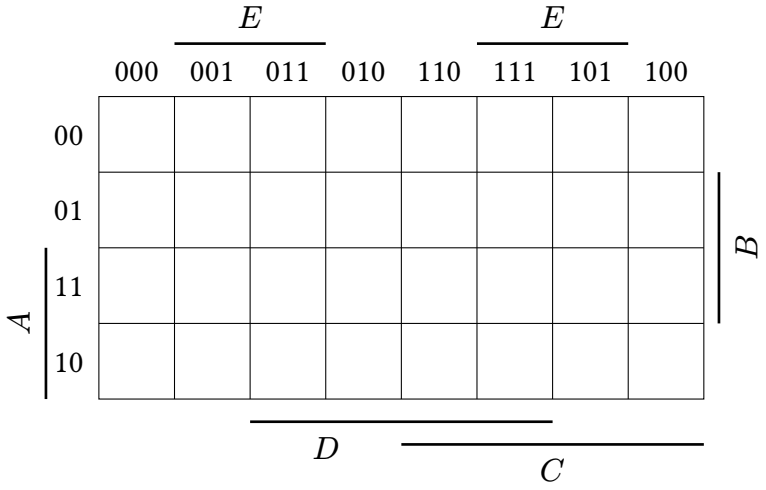
1. Вычислите[1] хеш SHA-256 h строки $s =$ «DM Fall 2025 HW4» (без кавычек).
2. Разделите h на восемь 32-битных блоков: $h = h_0 \parallel h_1 \parallel \ldots \parallel h_7$ где каждый h_i имеет длину 32 бита.
3. Примените операцию XOR ко всем блокам: $d = h_0 \oplus h_1 \oplus \cdots \oplus h_7$.
4. Примените маску: $w = d \oplus$ `0x71be8976`, получив 32-битный результат $w = (w_0 w_1 \ldots w_{31})_2$.

Полученная битовая строка w кодирует булеву функцию от 5 переменных $f^{(5)}$: бит w_0 (старший бит, MSB) представляет $f(0,0,0,0,0)$, и бит w_{31} (младший бит, LSB) представляет $f(1,1,1,1,1)$.

Проверка: Хеш h заканчивается на ...`01100010`; значение d (до наложения маски) начинается с `0000`...

b) В попытках освоить K-Map

Постройте карту Карно для 5 переменных для вашей функции, используя шаблон ниже:





[1] Для вычисления самих хешей SHA-256 можете использовать что угодно.

c) Нахождение минимальных форм

1. Найдите минимальные ДНФ и КНФ для f с помощью вашей карты Карно.
2. Подсчитайте все простые импликанты (prime implicants).
3. Если входы, где $A \wedge B \wedge C = 1$ это *don't-care conditions*[2] , как изменятся минимальные формы?

d) Анализируем результат

1. На сколько литералов меньше использует ваша минимальная ДНФ по сравнению с СДНФ?
2. Что для вашей функции более компактно: ДНФ или КНФ? Почему?

2. Анализ булевых функций

Рассмотрим следующие четыре функции:[3]:

1. $f_1 = f^{(4)}_{47541}$
2. $f_2 = \sum m(1, 4, 5, 6, 8, 12, 13)$
3. $f_3 = f^{(4)}_{51011} \oplus f^{(4)}_{40389}$
4. $f_4 = A\overline{B}D + \overline{A}\,\overline{C}D + \overline{B}C\overline{D} + A\overline{C}D$

a): Разложние булевых функций

1. Разложить до СДНФ по Шеннону функцию *3.* (не строя таблицу истинности).
2. Построить СКНФ функции *3.* по известной СДНФ

b): K-Maps

Постройте карту Карно для каждой функции.

c): Простые импликанты

1. Перечислите все импликанты функции *4.*
2. При помощи карты Карно найдите все простые импликанты функции *4.* (каноническая форма Блейка)[4]
3. При помощи метода Квайна-МакКласки найдите все простые импликанты функции *4.* (каноническая форма Блейка)
4. Определите, какие из них являются ядерными (EPI).

d): Минимальные ДНФ и КНФ

1. Используя метод Петрика (на уже полученном результате метода Квайна-МакКласки) найдите минимальную ДНФ функции .
2. Используя алгебраический метод найдите минимальную КНФ функции *4.*

[2]Недоопределенная функция

[3]Обозначение: $f^{(n)}_k$ это k-я булева функция от n переменных, где k десятичное значение таблицы истинности, которое, представленное в двоичном виде как $(f_0 f_1 ... f_{2^n-1})_2$, соответствует значениям f: MSB f_0 is $f(0, ..., 0)$, LSB f_{2^n-1} is $f(1, ..., 1)$.

[4]Сокращенная ДНФ - каноническая форма Блейка

e) Алгебраическая нормальная форма

1. Постройте АНФ для функции *1.* используя метод Паскаля.
2. Постройте АНФ для функции *2.* используя метод треугольника.
3. Постройте АНФ для функции *4.* используя K-map.

f) Полином Жегалкина

1. Определите степень полинома каждой функции из части (e).
2. Какая функция имеет наивысшую степень? Является ли она максимальной для этого количества переменных?
3. Докажите, что $\deg(f \oplus g) \leq \max(\deg(f), \deg(g))$. Найдите пример со строгим неравенством.

3. Коды Рида–Маллера

(необязательное, +1 доп. балл)

В 1971 году Маринер-9 передал первые снимки Марса крупным планом, используя коды Рида–Маллера. Эти помехоустойчивые коды построены из булевых функций с ограниченной степенью полинома.

Код Рида–Маллера $\mathrm{RM}(r, m)$ состоит из всех булевых функций $f : \mathbb{F}_2^m \to \mathbb{F}_2$ у которых $\deg(f) \leq r$.

Параметры: длина $n = 2^m$, размерность $k = \sum_{i=0}^{r} \binom{m}{i}$, минимальное расстояние Хэмминга[5] $d = 2^{m-r}$.

Построение порождающей матрицы: Строки соответствуют мономам степени $\leq r$. Столбцы соответствуют входамs $(x_1, ..., x_m) \in \mathbb{F}_2^m$. Элемент (i, j) — это значение монома i вычисленное на входе j.

Example: $\mathrm{RM}(1, 2)$ имеет мономы $\{1, x_1, x_2\}$ и входы $\{(0,0), (0,1), (1,0), (1,1)\}$:

$$G = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{pmatrix}$$

Сообщение $(1, 0, 1)$ кодируется в $(1, 0, 1, 0)$ через $\boldsymbol{c} = \boldsymbol{u}G$.

a) Построение кода

Рассмотрим $\mathrm{RM}(1, 3)$:

1. Вычислите параметры кода: длина n, размерность k, минимальное расстояние d, и способность исправления ошибок $t = \left\lfloor \frac{d-1}{2} \right\rfloor$.

[5] *Расстояние Хэмминга* между двумя кодовыми словами — это количество позиций, в которых они различаются.

2. Постройте порождающую матрицу G (4×8). Строки соответствуют мономам $\{1, x_1, x_2, x_3\}$. Столбцы упорядочены: $(0,0,0), (0,0,1), ..., (1,1,1)$, где столбец i (for $i = 1, ..., 8$) соответствует двоичному представлению числа $i - 1$.
3. Закодируйте сообщение $\boldsymbol{u} = (1,0,1,1)$ в кодовое слово $\boldsymbol{c} = \boldsymbol{u}G$.

b) Исправление одиночной ошибки

1. Инвертируйте *один* бит в позиции $i \in \{1, ..., 8\}$ вашего кодового слов $\boldsymbol{c}$ чтобы смоделировать ошибку передачи, получив принятое слово $\boldsymbol{r} = (r_1, ..., r_8)$.

2. Декодируйте для восстановления коэффициентов АНФ (a_0, a_1, a_2, a_3):
   - Для каждого a_j где $j > 0$: определите все пары позиций (p, q) чьи входные векторы различаются только переменной x_j Каждая пара дает *голос* $v = r_p \oplus r_q$. Установите $a_j = \text{majority}(v_1, v_2, ...)$.
   - Для a_0: после определения a_1, a_2, a_3, вычислите $a_0 = r_1 \oplus (a_1 \cdot 0) \oplus (a_2 \cdot 0) \oplus (a_3 \cdot 0) = r_1$, так как позиция 1 соответствует входу $(0,0,0)$.[6]

3. Убедитесь, что раскодированное сообщение (a_0, a_1, a_2, a_3) равно вашему исходному $\boldsymbol{u} = (1,0,1,1)$.

4. Покажите подробное вычисление для коэффициента a_1: перечислите все пары, различающиеся по x_1, вычислите каждый XOR-голос и определите большинство.

[6] В случае ничьей в голосовании выберите 0 в качестве значения по умолчанию. Заметьте, что при одной ошибке для $\text{RM}(1,3)$ ничьих возникать не должно..